MEDAC

Instituto Oficial de Formación Profesional

TEMA 1. EL SOFTWARE

TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

Curso 2023-2024

HARDWARE Y SOFTWARE

Dispositivos físicos, ejecutan el software.

Programas que permiten al usuario interactuar con el hardware.

FUNCIÓN

Realizar tareas a nivel de máquina.

FUNCIÓN

Dar instrucciones al hardware.

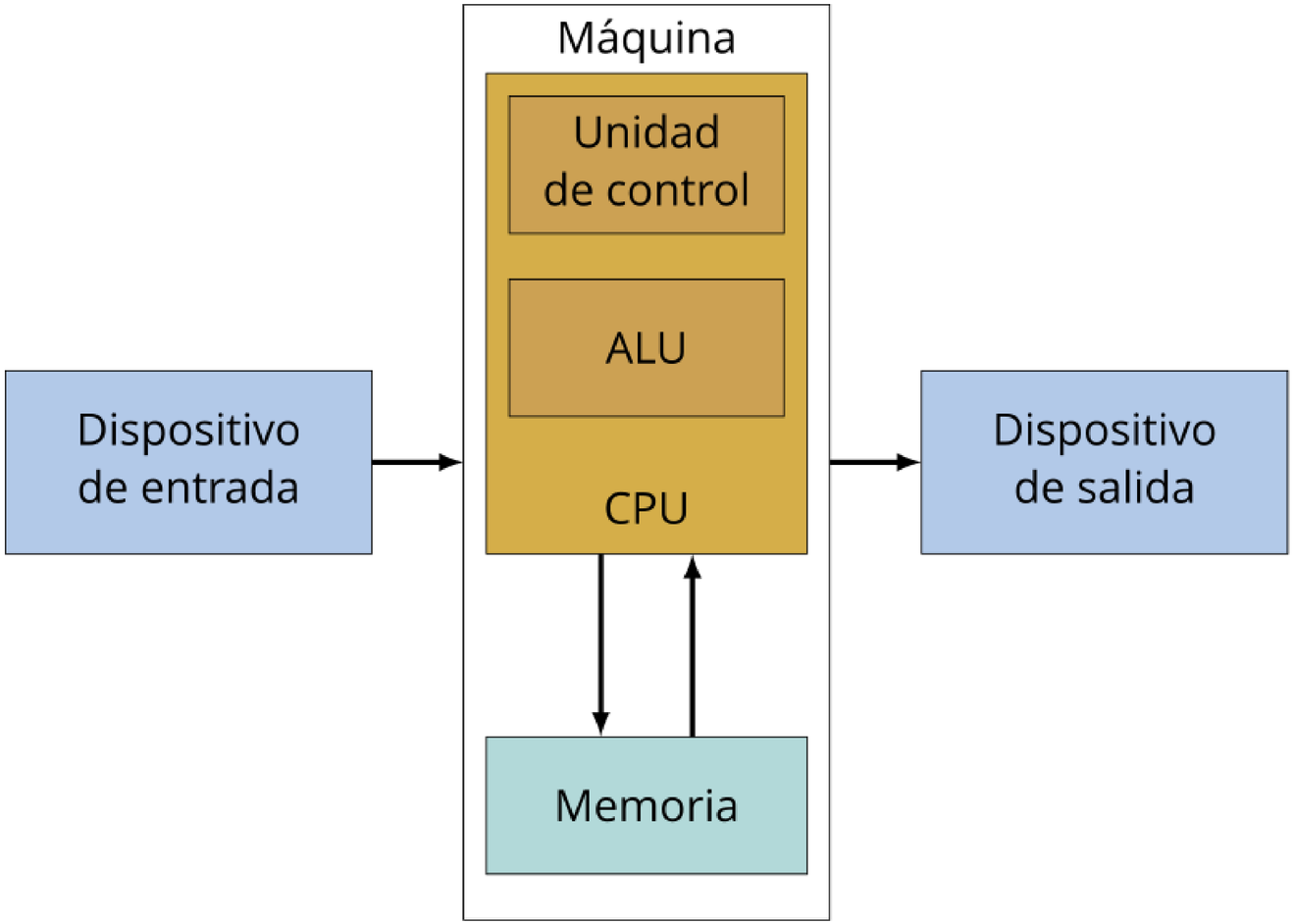
TIPOS DE HARDWARE

- Procesamiento.
- Almacenamiento.
- Periféricos de entrada y salida.

TIPOS DE SOFTWARE

- Software del sistema.
- Software de desarrollo.
- Software de aplicación.

ARQUITECTURA VON NEUMANN: MEMORIA + E/S + CPU



PROGRAMA INFORMÁTICO

- Receta detallada de cómo resolver un problema determinado.
- No puede ser ambiguo.
- Debe especificar la secuencia de instrucciones lógicas necesarias para llegar a la solución.

SOFTWARE INFORMÁTICO

- Conjunto de programas informáticos que se combinan entre sí para un determinado fin.
- Ejemplos: App de facturas , SO,..

ALGORITMO

- Secuencia de instrucciones.

ALGORITMO VS. PROGRAMA

ALGORITMO

- Secuencia de instrucciones
- No tiene por qué ser informático

PROGRAMA INFORMÁTICO

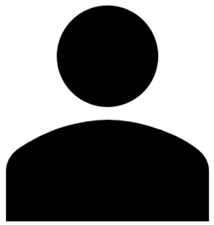
- Codificación detallada
- Instrucciones + Datos

LENGUAJE DE PROGRAMACIÓN

- Conjunto de sentencias y expresiones que permiten la comunicación programador – máquina.
- Lenguaje natural (Traducción) Lenguaje máquina = Compilación

NIVEL DE ABSTRACCIÓN

- Capacidad de un individuo para obviar los detalles y aún así poder realizar las tareas recomendadas.

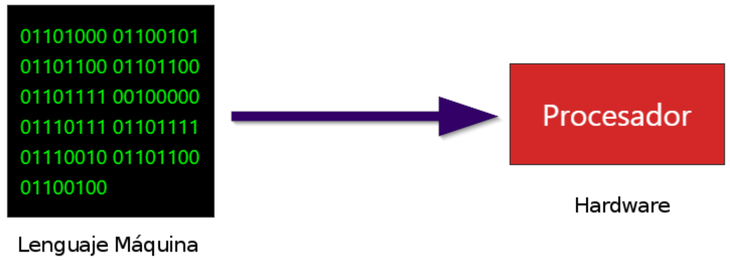


ALTO NIVEL (cerca del programador)

- Mayor legibilidad. Menor rendimiento. C#, Java

BAJO NIVEL (cerca de la computadora)

- Más difícil de entender. Mayor rendimiento. Ensamblador



NIVEL DE ABSTRACCIÓN

ABSTRACCIÓN: CAPACIDAD PARA OBVIAR DETALLES

ALTO NIVEL

```java
package es.javiergarciaescobedo.pongfx;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class Main extends Application {

    @Override
    public void start(Stage primaryStage)
        StackPane root = new StackPane();
        Scene scene = new Scene(root, 300,
        primaryStage.setTitle("PongFX");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

}
```

JAVA

NIVEL INTERMEDIO

```c
#include <stdio.h>
#define HT_FLAG         0x10000000
#define SSE_FLAG        0x02000000
#define SSE2_FLAG       0x04000000
#define SSSE3_FLAG      0x00000200
#define SSE4_1_FLAG     0x00080000
#define SSE4_2_FLAG     0x00100000

void cpuid(unsigned info, unsigned *eax, unsigned *ebx, unsigned *ecx, unsigned *edx)
{
    *eax = info;
    asm volatile(
        "mov %%ebx, %%edi;"
        "cpuid;"
        "mov %%ebx, %%esi;"
        "mov %%edi, %%ebx;"
        :"+a" (*eax), "=S" (*ebx), "=c" (*ecx), "=d" (*edx)
        : :"edi");
}

void cpuid_parse(char *buf)
{
    unsigned eax, ebx, ecx, edx;
    int i=0;
    cpuid(1, &eax, &ebx, &ecx, &edx);
    i+=sprintf(buf+i, "CPUID Features(EAX=1): ECX=0x%08x EDX=0x%08x\n", ecx, edx);
    if (edx & HT_FLAG) i+=sprintf(buf+i, "HT: YES\n"); else i+=sprintf(buf+i, "HT: NO
    if (edx & SSE_FLAG) i+=sprintf(buf+i, "SSE: YES\n"); else i+=sprintf(buf+i, "SSE:
    if (edx & SSE2_FLAG) i+=sprintf(buf+i, "SSE2: YES\n"); else i+=sprintf(buf+i, "SSE
    if (ecx & SSSE3_FLAG) i+=sprintf(buf+i, "SSSE3: YES\n"); else i+=sprintf(buf+i, "S
    if (ecx & SSE4_1_FLAG) i+=sprintf(buf+i, "SSE4_1: YES\n"); else i+=sprintf(buf+i,
    if (ecx & SSE4_2_FLAG) i+=sprintf(buf+i, "SSE4_2: YES\n"); else i+=sprintf(buf+i,

}
```

C

BAJO NIVEL

```asm
        .file   "hola.c"
        .section        .rodata
.LC0:
        .string "%d"
        .text
        .globl  main
        .type   main, @function
main:
.LFB0:
        .cfi_startproc
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        subq    $16, %rsp
        movl    $3, %edx
        movl    $2, %eax
        addl    %eax, %edx
        ;; movl $.LC0, %eax
        ;; movl %edx, %esi
```

ASM

PARADIGMAS DE PROGRAMACIÓN

1. LENGUAJES IMPERTIVOS

- Crean algoritmos a través de un conjunto de instrucciones y expresiones del lenguaje.

2. LENGUAJES DECLARATIVOS

- Sólo especifican los valores que se esperan al final de un flujo, sin especificar qué realizar para llegar al resultado.

PARADIGMAS Y ENFOQUES DE PROGRAMACIÓN

Los paradigmas se pueden combinar.
Ejemplo: Videojuegos.

- Gráficos: bajo nivel
- Lógica de funcionamiento: alto nivel.
- IA: declarativo

Imperativo

¿**Cómo** resolver el problema?

- Programación modular
- Programación estructurada
- Orientada a eventos

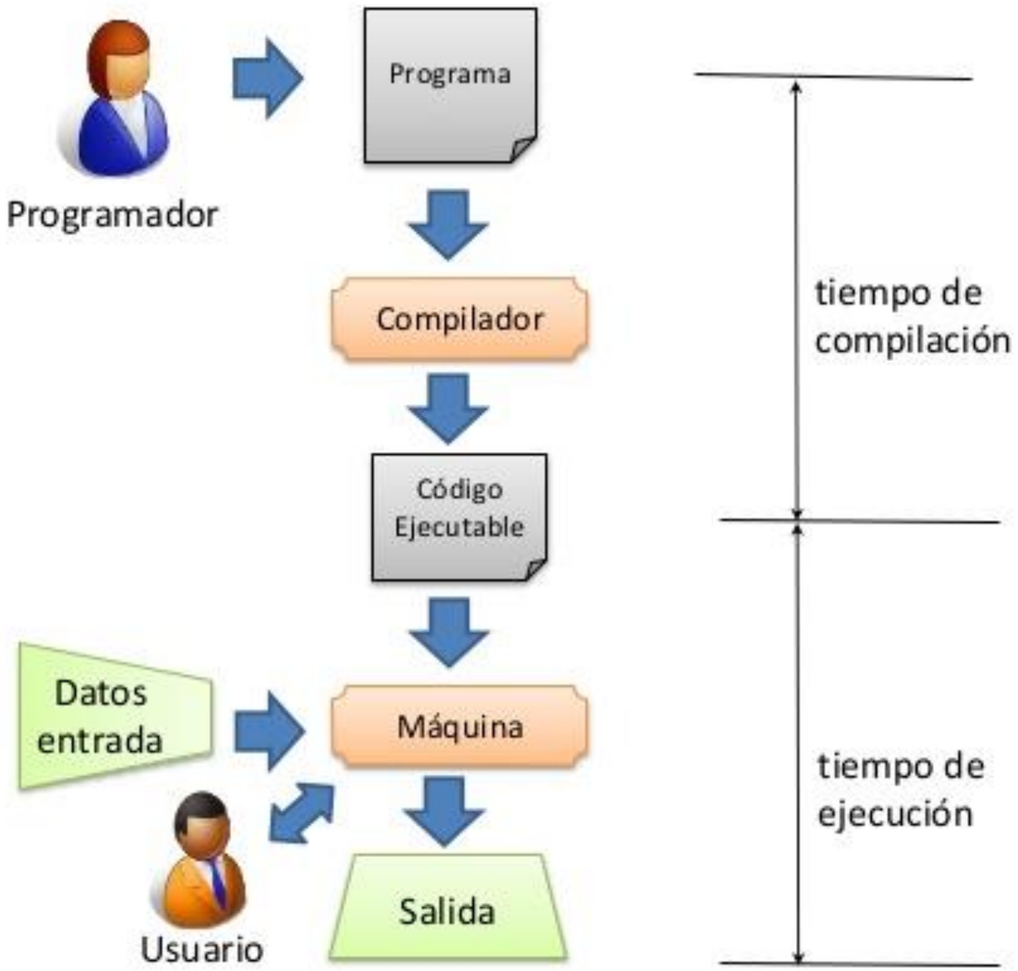
VS.

Declarativo

¿**Qué** hacer para resolver el problema?

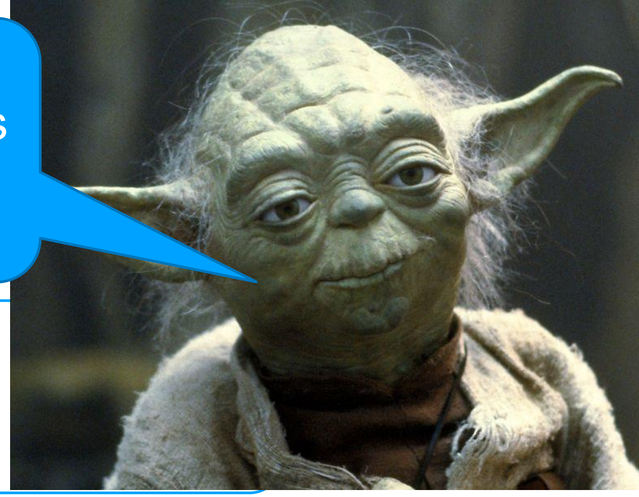
- Funcional
- Lógica
- Programación reactiva
- Lenguajes descriptivos

EL PROCESO DE COMPILACIÓN



EL PROCESO DE COMPILACIÓN: ETAPAS (I)

"Caminos a la victoria hay, distintos que aplastar a un enemigo"



Análisis Léxico

- Que las palabras estén bien escritas

Análisis Sintáctico

- Que las palabras estén en el orden correcto

Análisis Semántico

- Que las frases tengan sentido

LOS IDE HACEN EL ANÁLISIS EN TIEMPO REAL

EL PROCESO DE COMPILACIÓN: ETAPAS (II)

Código Intermedio
- Si el análisis es correcto.

→

Optimización
- (Opcional) Intenta mejorar el rendimiento.

→

Código objeto
- Fin del proceso. El código depende de la arquitectura

→

Código Ejecutable
- Enlaza código Objeto con Librerías

QUE SE COMPLETE LA COMPILACIÓN NO SIGNIFICA QUE EL PROGRAMA FUNCIONE

RESUMEN

- Niveles de **ABSTRACCIÓN**: **ALTO, MEDIO y BAJO** (Mayor legibilidad, menor rendimiento)

- **PARADIGMAS**: **IMPERATIVO** (Cómo) y **DECLARATIVO** (Qué)

- **Arquitectura** del computador: **modelo Von Neumann**
- Proceso de **COMPILACIÓN**: **Análisis + generación de código objeto**.

MEDAC

Instituto Oficial de Formación Profesional

- Muchas gracias -